MODUL PRAKTIKUM 07

MENGAMBIL DATA EKSTERNAL DENGAN REST API & RETROFIT

JSON • HTTP Client • Retrofit • Remote Data Source • Loading & Error State

Pemrograman Aplikasi Perangkat Bergerak (CIF62041)
S1 Teknik Informatika · Fakultas Ilmu Komputer · Universitas Brawijaya



Durasi Praktikum: 170 menit | Tahun 2026

Tim Pengelola Mata Kuliah

1. Identitas dan Capaian Praktikum

Modul ini melanjutkan project NavigationLab yang telah direfactor pada Modul Praktikum 06. Sumber data in-memory diganti dengan layanan REST API eksternal agar mahasiswa memahami bagaimana aplikasi Android mengambil JSON melalui HTTP client, mengubahnya menjadi object Kotlin, dan meneruskan hasil tersebut melalui data layer hingga UI.

Sub-CPMK terkait
Mahasiswa mampu mengambil, mengolah, dan menampilkan data eksternal melalui REST API, JSON, HTTP client, dan/atau Firebase. Pada modul ini fokus diarahkan pada REST API + Retrofit; Firebase tidak digunakan.

Capaian Praktikum

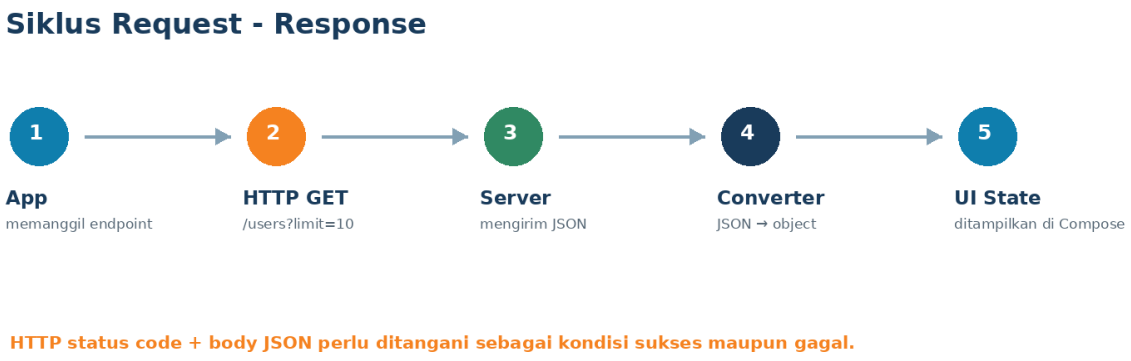
Setelah praktikum, mahasiswa mampu mengonfigurasi akses jaringan Android, mendefinisikan endpoint Retrofit, memetakan JSON ke DTO, mengakses data melalui remote data source dan repository, serta menampilkan kondisi loading, success, dan error pada Jetpack Compose.

Indikator Keberhasilan

- Aplikasi memiliki permission INTERNET dan dapat mengakses endpoint HTTPS.
- Retrofit mengubah endpoint REST menjadi fungsi Kotlin bertipe Call<T>.
- JSON response dipetakan ke DTO, lalu diubah ke model yang digunakan aplikasi.
- Composable UI tidak memanggil Retrofit secara langsung.
- Repository tetap menjadi pintu akses data bagi ViewModel.
- UI menampilkan kondisi loading, data berhasil, dan pesan error secara terpisah.
- Navigation Home → Detail → Back tetap berfungsi setelah sumber data dipindahkan ke network.

3. Konsep Singkat REST API, HTTP, dan JSON

REST API menyediakan resource melalui URL (endpoint). Aplikasi mengirim HTTP request, server memproses request, lalu mengirim HTTP response. Body response sering menggunakan JSON. Pada Android, HTTP client seperti Retrofit membantu mendefinisikan endpoint sebagai interface Kotlin dan mengonversi JSON menjadi object.



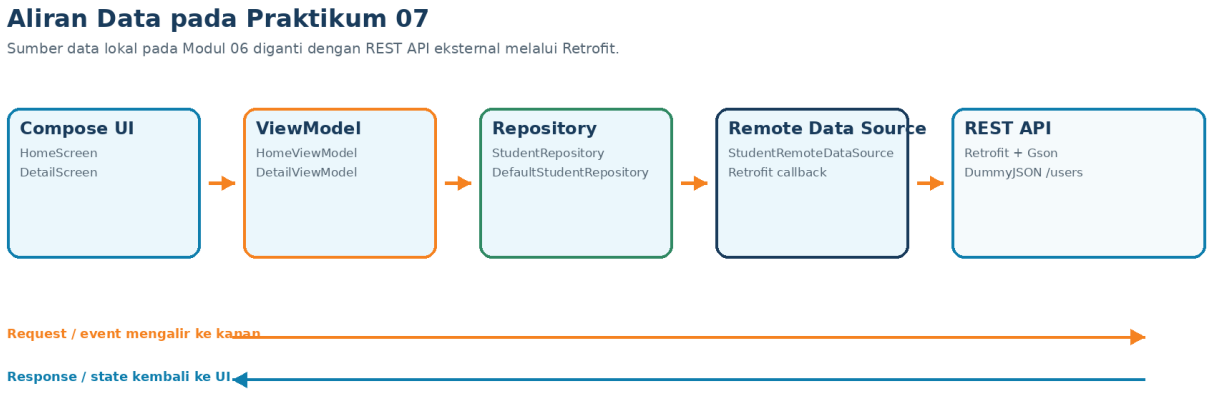
Gambar 1. Siklus request-response pada aplikasi Android.

Komponen yang Digunakan

Bagian	Keterangan
REST endpoint	URL yang merepresentasikan resource, misalnya GET /users.
HTTP method	GET digunakan untuk mengambil data. Modul ini belum membahas POST/PUT/DELETE.
JSON	Format pertukaran data berbasis pasangan key-value dan array.
Retrofit	Type-safe HTTP client yang mengubah interface Kotlin menjadi request HTTP.
Converter Gson	Mengonversi JSON response menjadi object DTO Kotlin.
DTO	Data Transfer Object; bentuk data yang mengikuti struktur response API.
Repository	Mengabstraksi asal data agar ViewModel tidak bergantung langsung pada Retrofit.

Catatan pedagogis
Modul ini sengaja menggunakan Retrofit callback (Call.enqueue) agar fokus pada REST, JSON, dan arsitektur data. Pada modul berikutnya implementasi network akan direfactor menggunakan Kotlin Coroutines dan Flow.

4. Arsitektur Setelah Menambahkan Remote Data



Gambar 2. Aliran data NavigationLab setelah sumber data dipindahkan ke REST API.

Prinsip utama tetap sama seperti Modul 06: UI hanya merender state dan mengirim event. ViewModel meminta data melalui repository. Repository memilih sumber data. Detail teknis Retrofit berada pada remote data source/network layer, bukan pada composable.

5. Langkah Kerja

Langkah 1 — Siapkan Project dari Modul 06

1. Buka project NavigationLab hasil Modul Praktikum 06.
2. Jalankan aplikasi dan pastikan Home, Detail, Profile, dan About masih berfungsi.
3. Pastikan Home menampilkan daftar mahasiswa dari repository lokal.
4. Buat checkpoint/commit sebelum mengubah data layer.

Langkah 2 — Tambahkan Permission Internet

Tambahkan permission berikut sebelum tag <application> pada AndroidManifest.xml. INTERNET adalah normal permission sehingga tidak memerlukan dialog runtime permission.

AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <uses-permission android:name="android.permission.INTERNET" />

    <application
        ... >
        ...
    </application>
</manifest>
```

Langkah 3 — Tambahkan Dependency Retrofit

Pada app/build.gradle.kts tambahkan Retrofit dan Gson converter. Versi 3.0.0 digunakan pada contoh ini.

app/build.gradle.kts

```
dependencies {
    // dependency Compose/ViewModel yang sudah ada ...

    implementation("com.squareup.retrofit2:retrofit:3.0.0")
    implementation("com.squareup.retrofit2:converter-gson:3.0.0")
}
```

Lakukan Gradle Sync. Jika dependency belum terunduh, pastikan komputer praktikum memiliki koneksi internet.

Langkah 4 — Uji Endpoint di Browser

Gunakan layanan DummyJSON sebagai REST API latihan. Buka endpoint berikut di browser untuk melihat JSON mentah:

Endpoint uji

```
https://dummyjson.com/users?limit=10
```

Amati field users, id, firstName, lastName, email, dan university. Kita hanya memetakan field yang dibutuhkan aplikasi.

Langkah 5 — Buat DTO untuk JSON

DTO sebaiknya berada di data/remote/dto karena strukturnya mengikuti kontrak API, bukan UI. Buat dua file berikut.

data/remote/dto/UserDto.kt

```
package com.example.navigationlab.data.remote.dto

data class UserDto(
    val id: Int,
    val firstName: String,
    val lastName: String,
    val email: String,
    val university: String
)
```

data/remote/dto/UserResponse.kt

```
package com.example.navigationlab.data.remote.dto

data class UserResponse(
    val users: List<UserDto>,
    val total: Int,
    val skip: Int,
    val limit: Int
)
```

Langkah 6 — Sesuaikan Model Student

Agar data dari API dapat ditampilkan tanpa membuat UI bergantung pada DTO, ubah model domain Student menjadi bentuk yang dibutuhkan aplikasi.

data/model/Student.kt

```
package com.example.navigationlab.data.model

data class Student(
    val id: Int,
    val name: String,
    val email: String,
    val university: String
)
```

Tambahkan mapper di file DTO agar bentuk data eksternal (UserDto) dapat diubah menjadi model internal aplikasi (Student).

Tambahan pada data/remote/dto/UserDto.kt

```
import com.example.navigationlab.data.model.Student

fun UserDto.toStudent() = Student(
    id = id,
    name = "$firstName $lastName",
    email = email,
    university = university
)
```

Langkah 7 — Definisikan Endpoint Retrofit

data/remote/StudentApiService.kt

```
package com.example.navigationlab.data.remote

import com.example.navigationlab.data.remote.dto.UserDto
import com.example.navigationlab.data.remote.dto.UserResponse
import retrofit2.Call
import retrofit2.http.GET
import retrofit2.http.Path
import retrofit2.http.Query

interface StudentApiService {

    @GET("users")
    fun getUsers(
        @Query("limit") limit: Int = 10
    ): Call<UserResponse>

    @GET("users/{id}")
    fun getUser(
        @Path("id") id: Int
    ): Call<UserDto>
}
```

Anotasi @GET menentukan HTTP method dan path endpoint. @Query menambahkan query parameter, sedangkan @Path mengganti bagian dinamis pada URL.

Langkah 8 — Buat Retrofit Client

data/remote/RetrofitClient.kt

```
package com.example.navigationlab.data.remote

import retrofit2.Retrofit
import retrofit2.converter.gson.GsonConverterFactory

object RetrofitClient {
    private const val BASE_URL = "https://dummyjson.com/"

    val apiService: StudentApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build()
            .create(StudentApiService::class.java)
    }
}
```

Perhatikan BASE_URL

Retrofit mengharuskan base URL diakhiri dengan karakter /. Path seperti users akan digabungkan dengan base URL saat request dibuat.

Langkah 9 — Buat Remote Data Source

Remote data source bertanggung jawab terhadap komunikasi dengan API. Ia mengubah response Retrofit menjadi Result agar repository menerima hasil sukses atau gagal dalam bentuk yang seragam.

data/remote/StudentRemoteDataSource.kt

```
package com.example.navigationlab.data.remote

import com.example.navigationlab.data.remote.dto.UserDto
import com.example.navigationlab.data.remote.dto.UserResponse
import retrofit2.Call
import retrofit2.Callback
import retrofit2.Response

class StudentRemoteDataSource(
    private val api: StudentApiService
) {
    fun getStudents(callback: (Result<List<UserDto>>) -> Unit) {
        api.getUsers().enqueue(object : Callback<UserResponse> {
            override fun onResponse(
                call: Call<UserResponse>,
                response: Response<UserResponse>
            ) {
                val body = response.body()
                if (response.isSuccessful && body != null) {
                    callback(Result.success(body.users))
                } else {
                    callback(Result.failure(Exception("HTTP ${response.code()}")))
                }
            }

            override fun onFailure(
                call: Call<UserResponse>,
                t: Throwable
            ) {
                callback(Result.failure(t))
            }
        })
    }

    fun getStudent(id: Int, callback: (Result<UserDto>) -> Unit) {
        api.getUser(id).enqueue(object : Callback<UserDto> {
            override fun onResponse(call: Call<UserDto>, response: Response<UserDto>) {
                val body = response.body()
                if (response.isSuccessful && body != null) {
                    callback(Result.success(body))
                } else {
                    callback(Result.failure(Exception("HTTP ${response.code()}")))
                }
            }

            override fun onFailure(call: Call<UserDto>, t: Throwable) {
                callback(Result.failure(t))
            }
        })
    }
}
```

Langkah 10 — Ubah Repository Menjadi Asinkron Berbasis Callback

Repository tetap menjadi kontrak akses data bagi ViewModel. Karena network tidak memberikan hasil secara instan, fungsi repository menerima callback.

data/repository/StudentRepository.kt

```
package com.example.navigationlab.data.repository

import com.example.navigationlab.data.model.Student

interface StudentRepository {
    fun getStudents(callback: (Result<List<Student>>) -> Unit)
    fun getStudent(id: Int, callback: (Result<Student>) -> Unit)
}
```

data/repository/DefaultStudentRepository.kt

```
package com.example.navigationlab.data.repository

import com.example.navigationlab.data.model.Student
import com.example.navigationlab.data.remote.StudentRemoteDataSource
import com.example.navigationlab.data.remote.dto.toStudent

class DefaultStudentRepository(
    private val remoteDataSource: StudentRemoteDataSource
) : StudentRepository {

    override fun getStudents(
        callback: (Result<List<Student>>) -> Unit
    ) {
        remoteDataSource.getStudents { result ->
            callback(result.map { users -> users.map { it.toStudent() } })
        }
    }

    override fun getStudent(
        id: Int,
        callback: (Result<Student>) -> Unit
    ) {
        remoteDataSource.getStudent(id) { result ->
            callback(result.map { it.toStudent() })
        }
    }
}
```

Langkah 11 — Tambahkan Loading dan Error pada HomeUiState

ui/home/HomeUiState.kt

```
package com.example.navigationlab.ui.home

import com.example.navigationlab.data.model.Student

data class HomeUiState(
    val students: List<Student> = emptyList(),
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)
```

Langkah 12 — Refactor HomeViewModel

ui/home/HomeViewModel.kt

```
class HomeViewModel(
    private val repository: StudentRepository
) : ViewModel() {

    var uiState by mutableStateOf(HomeUiState())
        private set

    init { loadStudents() }

    fun loadStudents() {
        uiState = uiState.copy(isLoading = true, errorMessage = null)

        repository.getStudents { result ->
            result
                .onSuccess { data ->
                    uiState = HomeUiState(students = data)
                }
                .onFailure { error ->
                    uiState = HomeUiState(
                        errorMessage = error.message ?: "Gagal mengambil data"
                    )
                }
        }
    }
}
```

Pola yang sama diterapkan pada DetailViewModel: set isLoading, panggil repository.getStudent(id), lalu ubah state berdasarkan Result.

Langkah 13 — Refactor DetailUiState dan DetailViewModel

ui/detail/DetailUiState.kt

```
data class DetailUiState(
    val student: Student? = null,
    val isLoading: Boolean = false,
    val errorMessage: String? = null
)
```

Potongan DetailViewModel.kt

```
fun loadStudent(id: Int) {
    uiState = uiState.copy(isLoading = true, errorMessage = null)

    repository.getStudent(id) { result ->
        result
            .onSuccess { student ->
                uiState = DetailUiState(student = student)
            }
            .onFailure { error ->
                uiState = DetailUiState(
                    errorMessage = error.message ?: "Data tidak tersedia"
                )
            }
    }
}
```

Langkah 14 — Tampilkan Loading, Error, dan Data pada HomeScreen

UI perlu membedakan tiga kondisi utama. Jangan hanya menampilkan list karena request network dapat membutuhkan waktu atau gagal.

Potongan HomeScreen.kt

```
@Composable
fun HomeScreen(
    uiState: HomeUiState,
    onStudentClick: (Int) -> Unit,
    onRetry: () -> Unit
) {
    when {
        uiState.isLoading -> {
            Box(Modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
        }

        uiState.errorMessage != null -> {
            Column(
                modifier = Modifier.fillMaxSize().padding(24.dp),
                verticalArrangement = Arrangement.Center
            ) {
                Text(uiState.errorMessage)
                Button(onClick = onRetry) { Text("Coba Lagi") }
            }
        }

        else -> {
            LazyColumn {
                items(uiState.students) { student ->
                    TextButton(onClick = { onStudentClick(student.id) }) {
                        Text("${student.id} - ${student.name}")
                    }
                }
            }
        }
    }
}
```

Langkah 15 — Perbarui DetailScreen

Tampilkan name, email, dan university dari Student. Tambahkan UI loading dan error dengan pola yang sama. DetailScreen tetap tidak boleh mengimpor Retrofit, API service, atau remote data source.

Potongan konten DetailScreen

```
student?.let { data ->
    Text(data.name, style = MaterialTheme.typography.headlineSmall)
    Text("Email: ${data.email}")
    Text("University: ${data.university}")
}
```

Langkah 16 — Hubungkan Dependency pada NavigationLabApp

Potongan NavigationLabApp.kt

```
val remoteDataSource = remember {
    StudentRemoteDataSource(RetrofitClient.apiService)
}
val repository = remember {
    DefaultStudentRepository(remoteDataSource)
}

// Home destination
val homeViewModel = remember { HomeViewModel(repository) }
HomeScreen(
    uiState = homeViewModel.uiState,
    onStudentClick = { id -> navController.navigate(Routes.detail(id)) },
    onRetry = homeViewModel::loadStudents
)
```

Pada destination Detail, buat DetailViewModel dengan repository yang sama lalu panggil loadStudent(studentId). Pastikan route dan back stack dari Modul 05 tetap tidak berubah.

Langkah 17 — Jalankan dan Amati Logcat

1. Jalankan aplikasi pada emulator/perangkat yang memiliki koneksi internet.
2. Pastikan Home menampilkan data user dari DummyJSON, bukan data lokal lama.
3. Buka salah satu item untuk menuju Detail dan periksa data yang tampil.
4. Tekan Back dan pastikan kembali ke Home.
5. Matikan koneksi internet, buka ulang aplikasi, dan pastikan error state tampil tanpa crash.
6. Aktifkan internet lalu tekan Coba Lagi.

6. Pengujian Praktikum

Tandai setiap kondisi berikut sebelum mengerjakan tugas akhir.

- ☐ Gradle Sync berhasil setelah menambahkan Retrofit dan converter-gson.
- ☐ Aplikasi memiliki permission INTERNET.
- ☐ Endpoint /users berhasil memberikan data JSON.
- ☐ Home menampilkan minimal 10 profil dari API.
- ☐ Detail menampilkan name, email, dan university dari API.
- ☐ Loading indicator muncul saat data sedang diminta.
- ☐ Ketika internet dimatikan, aplikasi menampilkan error state dan tidak crash.
- ☐ Tombol Coba Lagi memanggil ulang request.
- ☐ Profile dan About tetap dapat dibuka.
- ☐ Back stack Home → Detail → Back tetap benar.
- ☐ HomeScreen dan DetailScreen tidak mengimpor Retrofit atau StudentApiService.

Screenshot yang Wajib Diambil

- Screenshot 1 — struktur package data/remote, data/repository, dan ui.
- Screenshot 2 — JSON endpoint /users di browser.
- Screenshot 3 — Home menampilkan data hasil REST API.
- Screenshot 4 — Detail menampilkan name, email, dan university.
- Screenshot 5 — error state ketika koneksi internet dimatikan.
- Screenshot 6 — aplikasi berhasil kembali ke Home setelah Detail.

7. Tugas Praktikum

Tambahkan fitur pencarian profil menggunakan endpoint search dari DummyJSON. Tujuan tugas adalah memastikan mahasiswa memahami bahwa endpoint, query parameter, repository, state, dan UI harus tetap terpisah sesuai layer.

Ketentuan Implementasi

1. Tambahkan fungsi searchUsers(@Query("q") query: String) pada StudentApiService untuk endpoint users/search.

2. Tambahkan fungsi searchStudents(query: String, callback: ...) pada remote data source dan repository.

3. Tambahkan searchQuery pada HomeUiState atau state terpisah yang sesuai.

4. Tambahkan TextField pada HomeScreen untuk memasukkan kata pencarian.

5. Tambahkan tombol Cari. Pencarian dilakukan hanya ketika tombol ditekan (tidak perlu debounce).

6. Saat query kosong, tombol Cari harus memuat kembali daftar awal.

7. Loading dan error state tetap digunakan pada proses pencarian.

8. Klik hasil pencarian tetap membuka Detail berdasarkan id.

9. UI tidak boleh memanggil StudentApiService atau RetrofitClient secara langsung.

10. Tambahkan minimal satu screenshot hasil pencarian pada laporan.

Petunjuk endpoint

```
GET https://dummyjson.com/users/search?q=John
```

8. Luaran dan Format Laporan

A. Source Code

- Kumpulkan project Android Studio lengkap atau repository sesuai mekanisme kelas.
- Project harus memuat DTO, Retrofit service, Retrofit client, remote data source, repository, ViewModel, dan UI yang telah diperbarui.
- Hapus data/source lokal lama bila sudah tidak digunakan, atau beri nama yang jelas jika dipertahankan untuk perbandingan.
- Pastikan project dapat di-build.

B. Screenshot

Sertakan enam screenshot wajib dari bagian pengujian dan minimal satu screenshot hasil pencarian pada tugas praktikum.

C. Laporan Singkat

Laporan cukup 2–3 halaman di luar cover. Gunakan struktur berikut:

Bagian	Keterangan
1. Tujuan	Ringkas tujuan implementasi REST API dan Retrofit.

Bagian	Keterangan
2. Arsitektur	Jelaskan aliran UI → ViewModel → Repository → Remote Data Source → REST API.
3. Implementasi	Jelaskan endpoint, DTO, repository, serta loading/error state yang dibuat.
4. Hasil Pengujian	Masukkan screenshot dan jelaskan hasil request, detail, error, retry, dan back stack.
5. Tugas Pencarian	Jelaskan endpoint search dan hasil implementasi.
6. Refleksi	Apa perbedaan data lokal dan data remote? Apa masalah yang muncul ketika network gagal?

9. Troubleshooting Singkat

Gejala	Pemeriksaan
UnknownHostException	Periksa koneksi internet emulator/perangkat dan permission INTERNET.
HTTP 404	Periksa base URL, path @GET, @Path, dan ID yang diminta.
Expected BEGIN_OBJECT / parsing error	Periksa apakah DTO sesuai struktur JSON yang diterima.
Data kosong	Periksa response body dan parameter limit/query.
Cleartext HTTP traffic not permitted	Gunakan endpoint HTTPS. DummyJSON pada modul ini menggunakan HTTPS.
UI tidak berubah	Pastikan ViewModel benar-benar mengubah Compose state setelah callback selesai.
Crash setelah model berubah	Perbarui semua pemakaian field Student dari program/year menjadi email/university.
Detail membuka ID salah	Periksa route detail/{studentId}, @Path id, dan nilai argument dari backStackEntry.

10. Ringkasan

Pada praktikum ini NavigationLab beralih dari sumber data in-memory ke REST API. Retrofit mendefinisikan request HTTP melalui interface Kotlin, Gson mengubah JSON menjadi DTO, remote data source menangani komunikasi API, repository memetakan DTO ke model aplikasi, ViewModel mengelola loading/success/error state, dan Compose UI hanya merender state. Arsitektur yang sudah dibangun pada Modul 06 tetap dipertahankan sehingga perubahan sumber data tidak menyebar ke seluruh aplikasi.

Jembatan ke Modul Berikutnya
Callback pada modul ini bersifat sementara untuk memisahkan fokus konsep network. Pada materi berikutnya, proses asinkron akan disederhanakan menggunakan Kotlin Coroutines dan Flow.

11. Referensi

[1] Android Developers. Get data from the internet / networking on Android. https://developer.android.com/develop/connectivity/network-ops/connecting (diakses Oktober 2026).

[2] Android Developers. Basic Android Kotlin Compose - Get data from the internet. https://developer.android.com/codelabs/basic-android-kotlin-compose-getting-data-internet (diakses Oktober 2026).

[3] Square. Retrofit - A type-safe HTTP client for Android and Java. https://square.github.io/retrofit/ (diakses Oktober 2026).

[4] Maven Central. com.squareup.retrofit2:retrofit 3.0.0. https://central.sonatype.com/artifact/com.squareup.retrofit2/retrofit (diakses Oktober 2026).

[5] DummyJSON. Users REST API documentation. https://dummyjson.com/docs/users (diakses Oktober 2026).

[6] Ghita, C. (2022). Kickstart Modern Android Development with Jetpack and Kotlin. Packt Publishing.

[7] RPS Pemrograman Aplikasi Perangkat Bergerak (CIF62041), FILKOM Universitas Brawijaya, 2026.